

CareerForge Career Preparation Platform - Project Report

Chapter 1: Introduction

1.1 Project Overview

CareerForge is a full-stack, web-based career preparation platform that helps students and early-career professionals prepare for technical interviews and job placements.

1.2 Problem Statement

Job seekers face difficulty transitioning from theoretical knowledge to practical, interview-ready skills; resources are fragmented and lack personalization.

1.3 Objectives

- Centralized platform for learning and interview preparation; - Coding practice with graded assessments; - Resume building and company application tools.

1.4 Scope of the Project

Includes user auth, courses, coding problems, mock interviews, admin console, and resume builder. Excludes scheduling live interviews and payment gateway.

1.5 Methodology

Agile iterative development with sprints, prototyping, feedback cycles, and CI automation.

1.6 Organization of Report

This document covers literature survey, system design, implementation, testing, results, and future scope.

Chapter 2: Literature Survey

2.1 Existing System

Existing platforms provide piecemeal functionality (coding platforms, MOOCs, job portals).

2.2 Limitations

Fragmentation, limited company-focused prep, and poor feedback loops.

2.3 Proposed System

A unified platform integrating learning, practice, assessments, and company application flows.

2.4 Comparative Analysis

CareerForge compares favorably by combining features of many platforms into one.

Chapter 3: System Analysis and Design

3.1 Requirement Analysis

Functional and non-functional requirements focused on auth, problems, judge, and scalability.

3.2 Functional Requirements

User registration, problem attempts, assessments, admin tools, resume builder.

3.3 Non-Functional Requirements

Scalability, performance, security, maintainability, and usability.

3.4 Use Case Diagram

Actors: User, Admin, Recruiter. Use cases: Login, Attempt Problem, Create Assessment, Manage Content.

3.5 Data Flow Diagram

High-level flows: User -> Auth -> APIs; Submission -> Judge -> Results -> Dashboard.

3.6 ER Diagram

Entities: User, Problem, Submission, Assessment, Company, Job. Diagram included as image.

3.7 System Architecture

React frontend, Spring Boot backend, judge workers, MySQL, optional Redis cache.

Chapter 4: Implementation

4.1 Technologies Used

HTML, CSS, JavaScript, React, Monaco, Java Spring Boot, MySQL, Maven, Node.js/npm.

4.2 Module Description

Auth, User/Profile, Course, Problem, Judge, Assessment, Admin Console, Company Module.

4.3 Database Design

Relational design with tables for users, problems, testcases, submissions, assessments, companies.

4.4 Screenshots of Implementation

Screenshots to be included; placeholders present.

4.5 Code Snippets

Reference to key modules and locations in the repo (backend/src, frontend/src).

Chapter 5: Testing

5.1 Test Plan

Unit, integration, frontend unit tests, E2E tests, security testing.

5.2 Test Cases

Sample test cases for signup, login, attempt problem, submit assessment.

5.3 Test Results

Run tests and summarize pass/fail counts (to be populated).

5.4 Bug Fixes and Validation

List and fix issues found during testing; examples included.

Chapter 6: Results and Discussion

6.1 Output Screenshots

Screenshots and sample outputs (placeholders).

6.2 Performance Analysis

Response times and judge scaling notes.

6.3 Discussion of Results

User feedback, pilot metrics, improvement areas.

Chapter 7: Conclusion and Future Enhancement

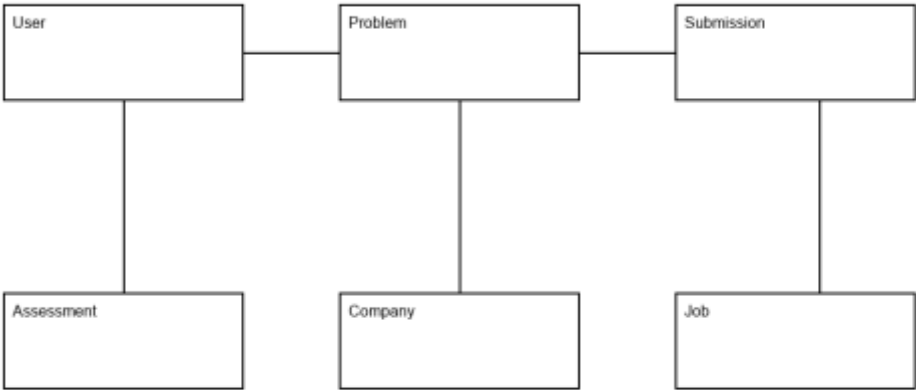
7.1 Conclusion

CareerForge provides integrated tools for preparation and recruiting; modular architecture supports future enhancements.

7.2 Future Scope

Add ML recommendations, video interviews, payment, gamification, multi-tenant support.

ER Diagram



Workflow Diagram

